

Bitácora S07

Mexi Banco: directorio, puerta de entrada y balanceo de carga

Sistemas Distribuidos 2027-1 · Grupo 2 · Facultad de Ingeniería, UNAM

Dato	Detalle
Alumna	Alejandra Melissa Saucedo Serralde
Práctica	Lab S07 (directorío Eureka, API gateway, tres réplicas de cuenta, matar una copia) y tarea (que transferencia también balancee por nombre)
Fecha	9 de octubre de 2026
Horas	Las horas de las capturas de terminal son hora local (UTC-6). El dashboard de Eureka muestra UTC, seis horas más: 21:22 local = 03:22 UTC.

1. Paso 1: el punto de partida, sin directorio

Levanté la versión base con `docker compose up --build -d`. Cada servicio publica su propio puerto (8081 a 8085) y las llamadas entre servicios usan URLs fijas, como `http://cuenta:8080`.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE
mexi-banco-v06-cuenta-1  mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta
About a minute ago    Up About a minute (healthy)        0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1      postgres:17-alpine                 "docker-entrypoint.s..."            db
About a minute ago    Up About a minute (healthy)        5432/tcp
mexi-banco-v06-movimiento-1  mexi-banco-v06-movimiento          "java -jar app.jar"                  movimiento
About a minute ago    Up About a minute (healthy)        0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1  mexi-banco-v06-notificacion        "java -jar app.jar"                  notificacion
About a minute ago    Up About a minute (healthy)        0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1      mexi-banco-v06-spei                "java -jar app.jar"                  spei
About a minute ago    Up About a minute (healthy)        0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1  mexi-banco-v06-transferencia      "java -jar app.jar"                  transferencia
About a minute ago    Up About a minute (healthy)        0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp
```

Figura 1. `docker compose ps` de la versión base: cada servicio publicado en su puerto (8081 a 8085).

No hay directorio todavía: a las 19:20:51 el puerto 8761 rechaza la conexión.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ date +%T
19:20:51
```

Figura 2. Hora de la prueba: 19:20:51.

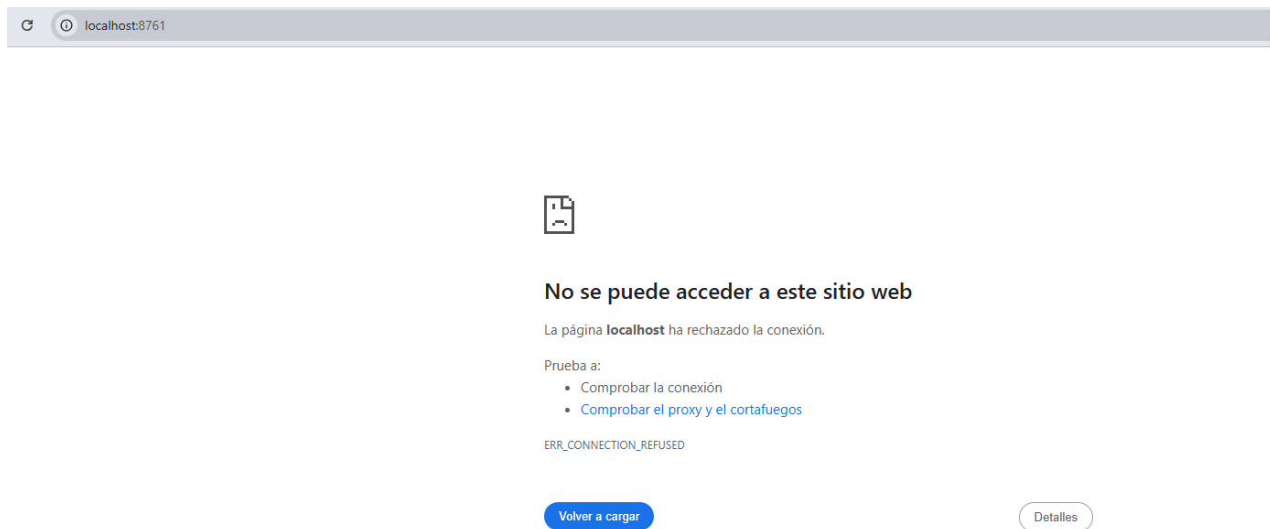


Figura 3. `http://localhost:8761` sin directorio: `ERR_CONNECTION_REFUSED`.

Como cada servicio está publicado, también puedo llamar directo a un endpoint interno desde afuera. Un POST `/movimientos` al puerto 8082 llega al servicio y responde 400 porque mandé el cuerpo vacío. Es decir, hoy nadie decide qué endpoints quedan expuestos.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 10 Oct 2026 01:31:42 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
```

Figura 4. POST directo a movimiento (8082): llega al servicio; el 400 es por el cuerpo vacío.

2. Paso 2: el directorio (Eureka), hito H1

Qué cambié

- `discovery/.../DiscoveryApplication.java`: agregué `@EnableEurekaServer`, que convierte la aplicación en el servidor de directorio.
- `discovery/src/main/resources/application.yml`: `register-with-eureka: false` y `fetch-registry: false`, para que el directorio no se registre consigo mismo. Los tiempos de barrido y de caché vienen de variables de entorno.
- `pom.xml` de `cuenta`, `movimiento`, `transferencia`, `notificacion` y `spei`: descomenté la dependencia del cliente de Eureka, que es lo que hace que cada servicio se registre y mande latidos.

- `docker-compose.yml`: descomenté el servicio `discovery` (puerto 8761, con `healthcheck`) y las dos líneas de `depends_on` para que los servicios arranquen hasta que el directorio esté sano.

Verificación

Los siete contenedores quedaron `healthy` y el dashboard muestra los cinco servicios en `UP` (1). El aviso rojo de “SELF PRESERVATION MODE IS TURNED OFF” es esperado: la autopreservación se apagó a propósito (`EUREKA_AUTOPRESERVACION=false`) para que una réplica muerta salga del registro en segundos y no en minutos. En el dashboard, “Renews (last min)” cuenta los latidos que llegan por minuto.

```
Meliss@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
mexi-banco-v06-cuenta-1             mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     2 minutes ago  Up About a m
inute (healthy) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1                 postgres:17-alpine                 "docker-entrypoint.s..."            db         53 minutes ago  Up 53 minute
s (healthy) 5432/tcp
mexi-banco-v06-discovery-1          mexi-banco-v06-discovery           "java -jar app.jar"                  discovery  2 minutes ago  Up 2 minutes
(healthy) 0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-movimiento-1         mexi-banco-v06-movimiento          "java -jar app.jar"                  movimiento 2 minutes ago  Up About a m
inute (healthy) 0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1      mexi-banco-v06-notificacion        "java -jar app.jar"                  notificacion 2 minutes ago  Up About a m
inute (healthy) 0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1              mexi-banco-v06-spei                "java -jar app.jar"                  spei        2 minutes ago  Up About a m
inute (healthy) 0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1     mexi-banco-v06-transferencia       "java -jar app.jar"                  transferencia 2 minutes ago  Up About a m
inute (healthy) 0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp
```

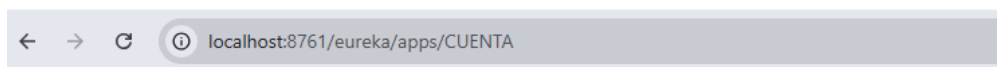
Figura 5. `docker compose ps`: `db`, `discovery` y los cinco servicios en `healthy`.

The screenshot shows the Eureka dashboard in a web browser. At the top, the system status is displayed with a red warning: "THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS." Below this, the "DS Replicas" section shows "localhost". The "Instances currently registered with Eureka" section contains a table with five rows, each representing a service: CUESTA, MOVIMIENTO, NOTIFICACION, SPEI, and TRANSFERENCIA. Each row shows the application name, AMIs, Availability Zones, and Status (UP (1)).

Application	AMIs	Availability Zones	Status
CUESTA	n/a (1)	(1)	UP (1) - c8dd8a636cd2.cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - d600c58feec5.movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 1aec39c5398f.notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 8dcf1c0bf5f0.spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 53bab962a60f.transferencia:8080

Figura 6. Dashboard de Eureka (Current time 01:59:50 UTC): cinco servicios en `UP` (1) y el aviso rojo de autopreservación apagada.

El XML de `CUESTA` muestra cómo ve Eureka a la instancia: `status UP`, la IP del contenedor, `vipAddress cuenta`, y los tiempos de latido (`renewalIntervalInSecs 5`, `durationInSecs 15`).



This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<application>
  <name>CUENTA</name>
  <instance>
    <instanceId>c8dd8a636cd2:cuenta:8080</instanceId>
    <hostName>172.22.0.6</hostName>
    <app>CUENTA</app>
    <ipAddr>172.22.0.6</ipAddr>
    <status>UP</status>
    <overriddenstatus>UNKNOWN</overriddenstatus>
    <port enabled="true">8080</port>
    <securePort enabled="false">443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
      <name>MyOwn</name>
    </dataCenterInfo>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791597490711</registrationTimestamp>
      <lastRenewalTimestamp>1791597633021</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791597488168</serviceUpTimestamp>
    </leaseInfo>
    <metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.22.0.6:8080</homePageUrl>
    <statusPageUrl>http://172.22.0.6:8080/actuator/info</statusPageUrl>
    <healthCheckUrl>http://172.22.0.6:8080/actuator/health</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791597490711</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791597487658</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
</application>
```

Figura 7. `http://localhost:8761/eureka/apps/CUENTA`: la instancia registrada, con latido cada 5 s y expiración de 15 s.

¿Alguien usa el directorio para encontrar a cuenta?

No, todavía nadie. Los servicios se registran y mandan latidos, así que el directorio ya sabe quién existe. Pero las llamadas entre servicios no lo consultan: `transferencia` y `spei` siguen usando `CUENTA_URL`: `http://cuenta:8080`, una URL escrita a mano en la que el nombre `cuenta` lo resuelve el DNS interno de Docker, no Eureka. Hoy Eureka es una lista que se llena y que nadie lee. Eso cambia con el gateway (`lb://cuenta`) y, en la tarea, con el balanceador de `transferencia`.

3. Paso 3: el gateway, hito H2

En `gateway/src/main/resources/application.yml` agregué las cuatro rutas que faltaban. Todas llevan `StripPrefix=1`, que quita `/api` antes de reenviar, y su destino es `lb://<servicio>`, que significa “pregúntale a Eureka por las instancias de este servicio y reparte entre ellas”.

Ruta pública	Método	Destino
/api/cuentas, /api/cuentas/{clabe}	todos	lb://cuenta
/api/movimientos	solo GET	lb://movimiento
/api/transferencias, /api/transferencias/{id}	todos	lb://transferencia
/api/notificaciones	solo GET	lb://notificacion
/api/spei	solo POST	lb://spei

Por eso los endpoints internos (POST /movimientos, POST /notificaciones, y /cargos y /abonos de cuenta) no tienen ruta: por la puerta dan 404. En el docker-compose.yml descomenté el servicio gateway, publicado en el 8080 y dependiente de discovery, y lo levanté con docker compose up -d --build gateway.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ docker compose ps gateway
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
mexi-banco-v06-gateway-1  mexi-banco-v06-gateway  "java -jar app.jar"    gateway    About a minute ago  Up About a minute (healthy)
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
```

Figura 8. El gateway quedó healthy en el puerto 8080.

Pruebas por la puerta

Una CLABE que no existe responde 404 de cuenta e incluye X-Instancia, el id del contenedor que contestó: la petición cruzó el gateway y llegó a cuenta. En el cuerpo, instance dice /cuentas/000 y no /api/cuentas/000, porque StripPrefix=1 ya quitó el prefijo.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: c8dd8a636cd2
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 02:09:00 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ |
```

Figura 9. GET /api/cuentas/000: 404 de cuenta, con X-Instancia: c8dd8a636cd2.

En cambio, POST /api/movimientos no tiene ruta. El 404 lo da el gateway, no hay X-Instancia y el error trae el formato propio del gateway (con requestId) en vez del Problem Details de cuenta.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-10T02:09:54.153+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"a0a80179-35"}
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
```

Figura 10. POST /api/movimientos: 404 sin X-Instancia; ninguna ruta lo atiende.

Con la CLABE s071791599910 creé una cuenta por la puerta (201) y la consulté (200). Las dos respuestas vinieron de la misma instancia, porque todavía había una sola copia de cuenta.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas -H "Content-Type: application/json" -d '{"clabe":"$LAB_CLABE","titular":"Ana","saldoInicial":1000}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: c8dd8a636cd2
Content-Type: application/json
Date: Sat, 10 Oct 2026 02:38:37 GMT

{"id":1,"clabe":"S071791599910","titular":"Ana","saldo":1000}
```

Figura 11. `POST /api/cuentas` por el gateway: 201 Created.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/$LAB_CLABE
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: c8dd8a636cd2
Content-Type: application/json
Date: Sat, 10 Oct 2026 02:39:32 GMT

{"id":1,"clabe":"S071791599910","titular":"Ana","saldo":1000.00}
```

Figura 12. `GET /api/cuentas/{clabe}` por el gateway: 200 OK.

4. Paso 4: tres réplicas de cuenta, hito H3

En el `docker-compose.yml` cambié `ports` de `cuenta` por `deploy: replicas: 3`. Tres contenedores no pueden publicar el mismo puerto 8081 de mi máquina, y ya no hace falta: desde afuera se llega por el gateway. También quité `ports` de `movimiento`, `transferencia`, `notificacion` y `spei`. Ahora las únicas entradas desde mi máquina son el gateway (8080) y el dashboard de Eureka (8761).

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ docker compose ps
```

NAME	STATUS	IMAGE	PORTS	COMMAND	SE
mexi-banco-v06-cuenta-1	Up About a minute (healthy)	mexi-banco-v06-cuenta	8080/tcp	"java -jar app.jar"	cu
mexi-banco-v06-cuenta-2	Up About a minute (healthy)	mexi-banco-v06-cuenta	8080/tcp	"java -jar app.jar"	cu
mexi-banco-v06-cuenta-3	Up About a minute (healthy)	mexi-banco-v06-cuenta	8080/tcp	"java -jar app.jar"	cu
mexi-banco-v06-db-1	Up 2 hours (healthy)	postgres:17-alpine	5432/tcp	"docker-entrypoint.s..."	db
mexi-banco-v06-discovery-1	Up 47 minutes (healthy)	sha256:d069c13b38543e05aa0eb1f916d5bd212de98a5668921df3aeb27e068fd54fbe	0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp	"java -jar app.jar"	di
mexi-banco-v06-gateway-1	Up 37 minutes (healthy)	mexi-banco-v06-gateway	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	"java -jar app.jar"	ga
mexi-banco-v06-movimiento-1	Up About a minute (healthy)	mexi-banco-v06-movimiento	8080/tcp	"java -jar app.jar"	mo
mexi-banco-v06-notificacion-1	Up About a minute (healthy)	mexi-banco-v06-notificacion	8080/tcp	"java -jar app.jar"	no
mexi-banco-v06-spei-1	Up About a minute (healthy)	mexi-banco-v06-spei	8080/tcp	"java -jar app.jar"	sp
mexi-banco-v06-transferencia-1	Up About a minute (healthy)	mexi-banco-v06-transferencia	8080/tcp	"java -jar app.jar"	tr

Figura 13. `docker compose ps`: `cuenta-1`, `cuenta-2` y `cuenta-3` en `healthy`, sin puerto publicado (solo 8080/tcp). Solo `gateway` y `discovery` publican puerto.

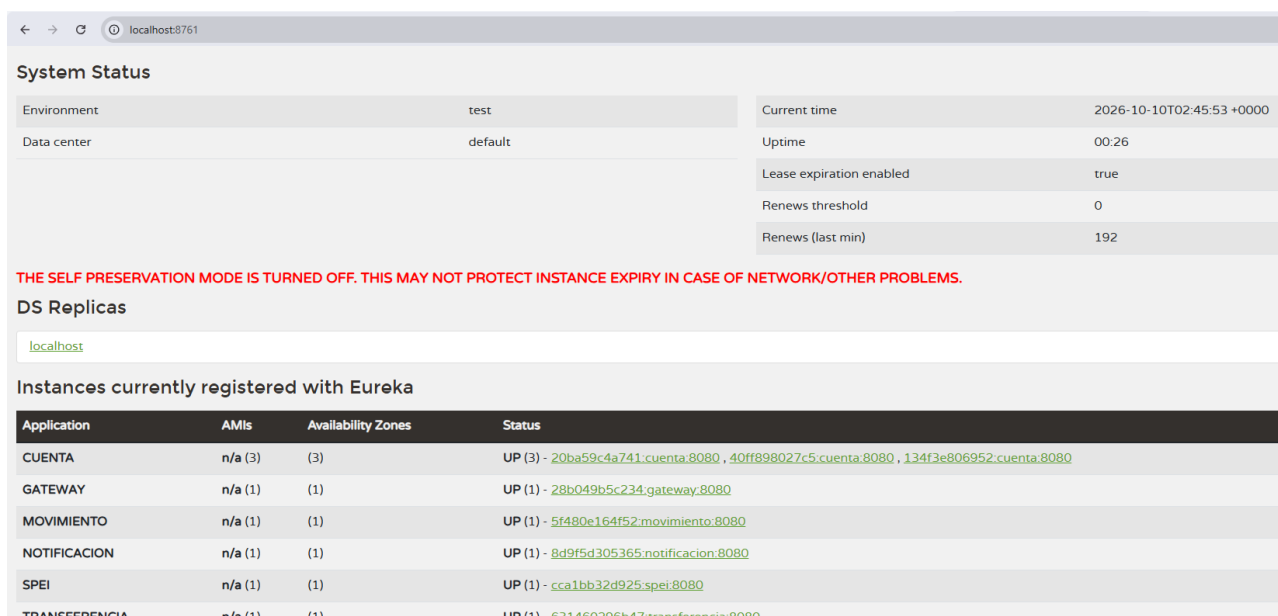


Figura 14. Dashboard: CUENTA en UP (3) con tres instancias (20ba59c4a741, 40ff898027c5, 134f3e806952).

Con nueve consultas seguidas por el gateway, las respuestas rotan entre las tres réplicas en el mismo orden (40ff..., 134f..., 20ba...), tres veces. Eso es round robin.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ for i in 1 2 3 4 5 6 7 8 9; do curl -s http://localhost:8080/api/cuentas/$LAB_CLABE | grep -i "x-instancia"; done
X-Instancia: 40ff898027c5
X-Instancia: 134f3e806952
X-Instancia: 20ba59c4a741
X-Instancia: 40ff898027c5
X-Instancia: 134f3e806952
X-Instancia: 20ba59c4a741
X-Instancia: 40ff898027c5
X-Instancia: 134f3e806952
X-Instancia: 20ba59c4a741
```

Figura 15. Nueve consultas a /api/cuentas/{clabe}: tres valores de X-Instancia en turnos.

5. Paso 5: matar una réplica

Predicción (escrita antes de correr el experimento)

Al matar una copia, Eureka no se entera al instante. Tiene que pasar la expiración del latido (15 s) más el barrido (5 s), y después el gateway tarda hasta 5 s en refrescar su lista y 5 s más por la caché del balanceador. Durante ese rato, el gateway seguirá mandando a la copia muerta una de cada tres peticiones, así que espero **unos 5 a 8 errores** durante **unos 15 a 25 segundos**. Después las peticiones se repartirán solo entre las dos copias vivas y no habrá más errores.

Cómo lo hice

En una terminal dejé un ciclo que consulta la cuenta por el gateway cada segundo y muestra la hora, el código HTTP y el X-Instancia. En una segunda terminal corrí `date +%T && docker kill mexi-banco-v06-cuenta-1`, y en el navegador seguí el dashboard de Eureka.

```
UCRT64:/c:/Users/Melis/mexi-banco-s07
21:21:47 HTTP/1.1 200 OK X-Instancia: 134f3e806952
21:21:48 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:21:49 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:21:50 HTTP/1.1 200 OK X-Instancia: 134f3e806952
21:21:51 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:21:53 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:21:54 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:21:55 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:21:56 HTTP/1.1 200 OK X-Instancia: 134f3e806952
21:21:57 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:21:59 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:00 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:01 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:02 HTTP/1.1 200 OK X-Instancia: 134f3e806952
21:22:03 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:04 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:06 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:07 HTTP/1.1 500 Internal Server Error
21:22:10 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:11 HTTP/1.1 500 Internal Server Error
21:22:14 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:15 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:17 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:18 HTTP/1.1 500 Internal Server Error
21:22:21 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:22 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:23 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
```

Figura 16. Terminal 1, antes y justo después del kill: respuestas 200 rotando entre tres réplicas y los primeros 500 (21:22:07, :11 y :18).

```
UCRT64:/c:/Users/Melis/mexi-banco-s07
Melis@DESKTOP-E1DRUJB UCRT64 ~
$ cd ~/mexi-banco-s07

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ date +%T && docker kill mexi-banco-v06-cuenta-1
21:22:02
mexi-banco-v06-cuenta-1

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$
```

Figura 17. Terminal 2: `date +%T && docker kill mexi-banco-v06-cuenta-1`, ejecutado a las 21:22:02.

```
UCRT64;c:/Users/Melis/mexi-banco-s07
21:22:22 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:23 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:24 HTTP/1.1 500 Internal Server Error
21:22:28 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:29 HTTP/1.1 500 Internal Server Error
21:22:32 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:33 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:34 HTTP/1.1 500 Internal Server Error
21:22:37 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:39 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:40 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:41 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:42 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:43 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:44 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:46 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:47 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:48 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:49 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:50 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:51 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:53 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:54 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:22:55 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
21:22:59 HTTP/1.1 200 OK X-Instancia: 20ba59c4a741
21:23:00 HTTP/1.1 200 OK X-Instancia: 40ff898027c5
```

Figura 18. Terminal 1 después del kill: los 500 de las 21:22:24, :29 y :34, y las respuestas limpias desde las 21:22:37.

Resultados

Hora local	Segundos desde el kill	Qué pasó
21:22:02	0	docker kill de cuenta-1 (instancia 134f3e806952). Su última respuesta fue a esta misma hora.
21:22:07	+5	Primer error 500.
21:22:32	+30	Eureka cancela la instancia (03:22:32 UTC en “Last 1000 cancelled leases”); CUENTA pasa de UP (3) a UP (2).
21:22:34	+32	Último error 500.
21:22:37	+35	Primera línea limpia: el gateway ya no le manda tráfico.

Hubo **6 errores**, todos 500 Internal Server Error, a las 21:22:07, :11, :18, :24, :29 y :34: uno de cada tres turnos, que es la parte del round robin que le tocaba a la réplica muerta. Los saltos de 3 o 4 segundos entre líneas son las peticiones fallidas, que esperan el tiempo máximo de conexión del gateway (2 s) antes de responder.

localhost:8761

System Status

Environment	test	Current time	2026-10-10T03:25:42 +0000
Data center	default	Uptime	01:06
		Lease expiration enabled	true
		Renews threshold	0
		Renews (last min)	168

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (2)	(2)	UP (2) - 20ba59c4a741-cuenta:8080 , 40ff898027c5-cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - 28b049b5c234-gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 5f480e164f52-movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 8d9f5d305365-notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - cca1bb3d925-spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 631460296b47-transferencia:8080

Figura 19. Dashboard después del kill (Current time 03:25:42 UTC): CUENTA en UP (2), sin 134f3e806952.

localhost:8761/lastn

System Status

Environment	test	Current time	2026-10-10T03:26:57 +0000
Data center	default	Uptime	01:08
		Lease expiration enabled	true
		Renews threshold	0
		Renews (last min)	168

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Last 1000 cancelled leases Last 1000 newly registered leases

Timestamp	Lease
10 oct 2026, 3:22:32 a. m.	CUENTA(134f3e806952:cuenta:8080)
10 oct 2026, 2:42:49 a. m.	TRANSFERENCIA(53bab962a60f:transferencia:8080)
10 oct 2026, 2:42:44 a. m.	NOTIFICACION(1aec39c5398f:notificacion:8080)
10 oct 2026, 2:42:39 a. m.	CUENTA(c8dd8a636cd2:cuenta:8080)
10 oct 2026, 2:42:34 a. m.	SPEI(8dcf1c0bf5f0:spei:8080)
10 oct 2026, 2:42:19 a. m.	MOVIMIENTO(d600c58fec5:movimiento:8080)
10 oct 2026, 2:38:14 a. m.	GATEWAY(28b049b5c234:gateway:8080)

Figura 20. "Last 1000 cancelled leases": CUENTA (134f3e806952:cuenta:8080) cancelada a las 3:22:32 a. m. UTC (21:22:32 local).

Contraste con la predicción

- **Número de errores: acertado.** Predije de 5 a 8 y fueron 6, uno de cada tres turnos.
- **Duración: me quedé corta.** Predije de 15 a 25 s y los errores duraron hasta los 32 s (35 s hasta la primera línea limpia).

La diferencia viene de que los retrasos se suman en cadena. Eureka tardó **30 s** en expulsar la instancia, no 15: la expiración está configurada en 15 s, pero se sabe que Eureka la cuenta de forma que en la práctica dura cerca del doble. Después el gateway tardó unos 5 s más en enterarse

(refresco de su lista de instancias y caché del balanceador, ambos de 5 s). Mientras eso pasaba, seguía mandando una de cada tres peticiones a una réplica que ya no existía.

Las peticiones que fallaron eran consultas (GET), así que no pasó nada grave. Si hubiera sido un cargo o un abono, un 500 así dejaría la duda de si la operación se aplicó o no.

Revivir la copia

Con `docker start mexi-banco-v06-cuenta-1` la réplica volvió a registrarse con el mismo id y CUENTA regresó a UP (3).

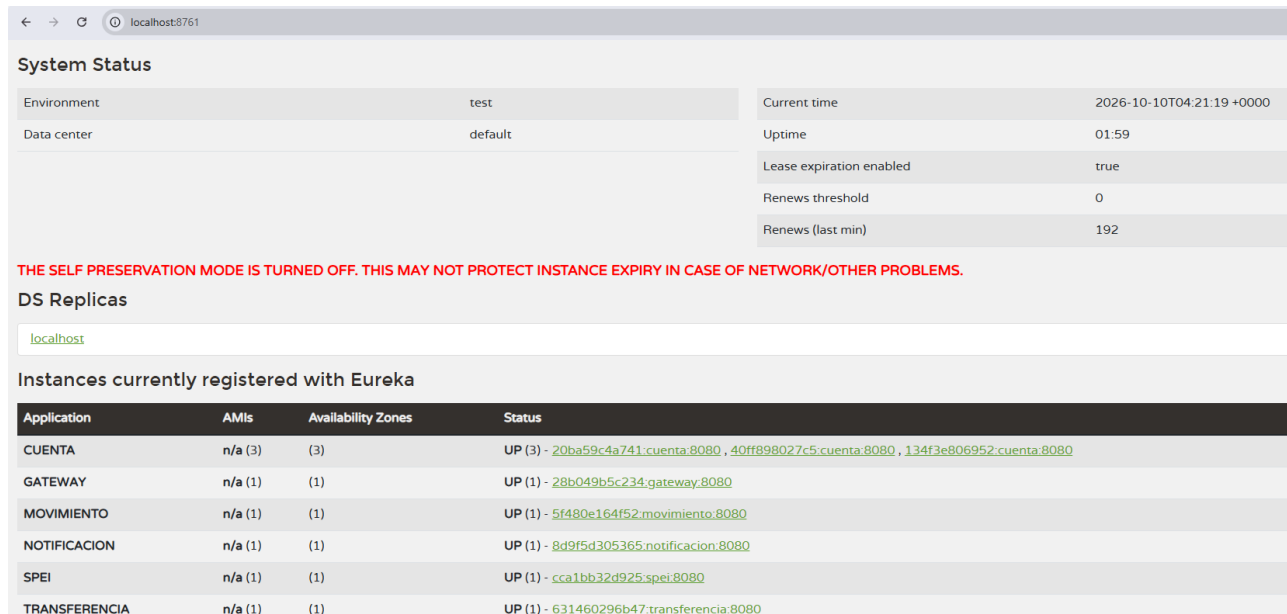


Figura 21. Dashboard con la réplica revivida: CUENTA otra vez en UP (3) (Current time 04:21:19 UTC).

6. Cierre: lo que resuelve cada pieza

El directorio resuelve quién existe y dónde está: guarda, por nombre, la lista de instancias vivas de cada servicio con su dirección, y las saca solo cuando dejan de mandar latidos.

El gateway resuelve quién entra y por dónde: es la única puerta hacia afuera (8080), traduce `/api/...` a cada servicio y deja sin ruta los endpoints internos, que dan 404.

El balanceador resuelve a cuál copia le toca cada petición: toma la lista del directorio y reparte por turnos (round robin) entre las copias de `cuenta`.

Cuando murió una copia vi 6 errores 500 durante unos 30 s **porque** el directorio tardó 30 s en expulsarla (los latidos dejaron de llegar) y el gateway tardó unos 5 s más en actualizar su lista; mientras tanto seguía mandándole una de cada tres peticiones a la copia muerta.

7. Tarea: que transferencia también balancee por nombre

Hasta aquí solo el gateway usaba el directorio. `transferencia` seguía llamando a `http://cuenta:8080`, una URL fija que el DNS de Docker resuelve sin consultar a Eureka. Hice dos cambios:

- transferencia/.../clientes/CientesHttp.java: agregué un `RestClient.Builder` marcado con `@LoadBalanced` (con los imports `RestClientBuilderConfigurer` y `LoadBalanced`). Con esa anotación, el host de la URL se toma como nombre de servicio y se cambia, en cada petición, por la IP y el puerto de una instancia registrada en Eureka.
- docker-compose.yml, servicio transferencia: CUENTA_URL, MOVIMIENTO_URL y NOTIFICACION_URL pasaron a `http://cuenta`, `http://movimiento` y `http://notificacion`, sin puerto. spei lo dejé igual, porque la tarea solo pide transferencia.

```
@Bean
@LoadBalanced
RestClient.Builder restClientBalanceado(RestClientBuilderConfigurer configurador) {
    return configurador.configure(RestClient.builder());
}
```

Reconstruí solo ese servicio con `docker compose up -d --build transferencia` y quedó healthy.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ docker compose ps transferencia
```

NAME	CREATED	STATUS	IMAGE	PORTS	COMMAND	SERVICE
mexi-banco-v06-transferencia-1	36 seconds ago	Up 30 seconds	mexi-banco-v06-transferencia	8080/tcp	"java -jar app.jar"	transferencia

Figura 22. *transferencia* reconstruida y healthy.

Luego corrí `./demo-v06.sh`, que hace una transferencia real por el gateway. La demo muestra a qué réplica de cuenta fue cada llamada de transferencia: la consulta del origen fue a cuenta-3, la del destino a cuenta-2, el cargo a cuenta-1 y el abono a cuenta-3. Cada llamada la reparte el balanceador de transferencia, no el DNS de Docker. La salida completa está en el Anexo A.

```
== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen": "002180009160741101", "claveDestino": "002180009160741102", "monto": 200, "fecha": "2026-11-01"}'
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-3 GET /cuentas/002180009160741101 atendida por 20ba59c4a741
cuenta-2 GET /cuentas/002180009160741102 atendida por 40ff898027c5
cuenta-1 POST /cuentas/002180009160741101/cargos atendida por 134f3e806952
cuenta-3 POST /cuentas/002180009160741102/abonos atendida por 20ba59c4a741
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia
```

Figura 23. Fragmento de la demo, paso 3: transferencia llamó a `http://cuenta` y cada llamada cayó en una réplica distinta.

Los saldos cuadran: el dinero no se crea ni se pierde ($1000 + 500 = 1500$ antes, $800 + 700 = 1500$ después). Después, un SPEI de 50 mandado dos veces con la misma `Idempotency-Key` se cobró una sola vez (Ana quedó en 750), y las nueve consultas seguidas volvieron a mostrar tres réplicas en turnos.

```

== 5. Saldos: el dinero no se crea ni se pierde ==
$ curl http://localhost:8080/api/cuentas/002180009160741101
{"id":2,"clabe":"002180009160741101","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009160741102
{"id":3,"clabe":"002180009160741102","titular":"Beto","saldo":700.00}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

```

Figura 24. Fragmento de la demo, paso 5: Ana 800 y Beto 700, suma 1500.

```

-> mismo id (1 -> 1): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ==
consulta 1 X-Instancia: 20ba59c4a741 (mexi-banco-v06-cuenta-3)
consulta 2 X-Instancia: 40ff898027c5 (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: 134f3e806952 (mexi-banco-v06-cuenta-1)
consulta 4 X-Instancia: 20ba59c4a741 (mexi-banco-v06-cuenta-3)
consulta 5 X-Instancia: 40ff898027c5 (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: 134f3e806952 (mexi-banco-v06-cuenta-1)
consulta 7 X-Instancia: 20ba59c4a741 (mexi-banco-v06-cuenta-3)
consulta 8 X-Instancia: 40ff898027c5 (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 134f3e806952 (mexi-banco-v06-cuenta-1)
-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas

```

Figura 25. Fragmento de la demo, pasos 7 y 8: el SPEI repetido no se cobra dos veces (Ana 750) y el round robin de nueve consultas.

8. Conclusiones

- Son tres piezas para tres problemas distintos: el directorio sabe quién existe, el gateway decide quién entra y por dónde, y el balanceador reparte entre las copias.
- El directorio no es instantáneo. La información viaja por varias cachés y relojes (latido, expiración, barrido, refresco del gateway, caché del balanceador), así que durante unos 30 s un cliente puede seguir llamando a algo que ya murió. Por eso hacen falta tiempos máximos y reintentos, tema que el código ya deja para S08.
- Los endpoints internos solo se esconden por la puerta: dentro de la red de Compose siguen abiertos. La demo lo deja anotado como pendiente de seguridad para más adelante.

Anexo A. Salida completa de ./demo-v06.sh

La demo corrió de principio a fin. La primera parte cubre los pasos 0 a 6 y la segunda los pasos 7 a 11.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)
$ cd ~/mexi-banco-s07
./demo-v06.sh

=== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ===
-> listo en 2 s. lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA      3 instancia(s)
GATEWAY      1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI         1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009160741101","titular":"Ana","saldoInicial":1000}'
{"id":2,"clabe":"002180009160741101","titular":"Ana","saldo":1000}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009160741102","titular":"Beto","saldoInicial":500}'
{"id":3,"clabe":"002180009160741102","titular":"Beto","saldo":500}
[HTTP 201]

=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160741101","claveDestino":"002180009160741102","monto":200}'
{"id":1,"claveOrigen":"002180009160741101","claveDestino":"002180009160741102","monto":200,"fecha":"2026-10-10T04:43:44.756467487Z"}
[HTTP 201]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-3 GET /cuentas/002180009160741101 atendida por 20ba59c4a741
cuenta-2 GET /cuentas/002180009160741102 atendida por 40ff898027c5
cuenta-1 POST /cuentas/002180009160741101/cargos atendida por 134f3e806952
cuenta-3 POST /cuentas/002180009160741102/abonos atendida por 20ba59c4a741
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===
$ curl http://localhost:8080/api/transferencias/1
{"id":1,"claveOrigen":"002180009160741101","claveDestino":"002180009160741102","monto":200.00,"fecha":"2026-10-10T04:43:44.756467Z"}
[HTTP 200]

=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009160741101
{"id":2,"clabe":"002180009160741101","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009160741102
{"id":3,"clabe":"002180009160741102","titular":"Beto","saldo":700.00}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clabe=002180009160741101
[{"id":4,"clabeCuenta":"002180009160741101","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"saldoResultante":800.00,"fecha":"2026-10-10T04:43:44.431774Z","referencia":"a 002180009160741102"}, {"id":2,"clabeCuenta":"002180009160741101","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-10T04:43:36.956351Z","referencia":"saldo inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clabe=002180009160741102
[{"id":2,"clabeCuenta":"002180009160741102","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-10T04:43:49.205133Z"}]
[HTTP 200]
```

Anexo A, parte 1: pasos 0 a 6.

```

=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91607411' -d '{"claveOrigen":"002180009160741101","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{"id":1,"idempotencyKey":"spei-demo-91607411","claveOrigen":"002180009160741101","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50,"estado":"ENVIADO"}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91607411' -d '{"claveOrigen":"002180009160741101","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{"id":1,"idempotencyKey":"spei-demo-91607411","claveOrigen":"002180009160741101","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
-> mismo id (1 -> 1): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
consulta 1 X-Instancia: 20ba59c4a741 (mexi-banco-v06-cuenta-3)
consulta 2 X-Instancia: 40ff898027c5 (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: 134f3e806952 (mexi-banco-v06-cuenta-1)
consulta 4 X-Instancia: 20ba59c4a741 (mexi-banco-v06-cuenta-3)
consulta 5 X-Instancia: 40ff898027c5 (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: 134f3e806952 (mexi-banco-v06-cuenta-1)
consulta 7 X-Instancia: 20ba59c4a741 (mexi-banco-v06-cuenta-3)
consulta 8 X-Instancia: 40ff898027c5 (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 134f3e806952 (mexi-banco-v06-cuenta-1)
-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009160741101/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T04:44:00.453+00:00","path":"/api/cuentas/002180009160741101/abonos","status":404,"error":"Not Found","requestId":"bbf7278b-2257"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160741101","claveDestino":"002180009160741199","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009160741199","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160741101","claveDestino":"002180009160741102","monto":999999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180009160741101","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s07 (trabajo-s07)

```

Anexo A, parte 2: pasos 7 a 11.